

Task 4.1: Bandit Static Analysis

Author: Nebil Müren - cas3322

Summary

Security analysis of the entire codebase using Bandit static analysis tool.

Code scanned:

- Total lines of code: 6448
- Total lines skipped (#nosec): 0

Run metrics:

- Total issues (by severity):
 - High: 2
 - Medium: 3
 - Low: 32
- Total issues (by confidence):
 - High: 30
 - Medium: 5
 - Low: 2

Issues and Mitigation

B324 - Weak Cryptographic Hash (HIGH Severity)

Issue: Use of weak SHA1 hash.

Mitigation: This is a supply chain issue. The usage is to check the 3rd party services which only accept SHA1 format. Doesn't require any developer action.

Affected files:

- flaskshop/account/utils.py:226

B602 (HIGH Severity) & B404, B603 (LOW Severity, 2 issues) - Subprocess Usage

Issue: Uses `subprocess.call()` with `shell=True`. Command injection vulnerability if user input reaches this code path.

Mitigation: Risk is negligible. Due to only using a static value, no user input is involved. Validate and sanitize all inputs before using.

Affected files:

- flaskshop/commands.py:5, 39, 66

B608 - Hardcoded SQL Expressions (MEDIUM Severity)

Issue: Uses string-based SQL query construction. Potential SQL injection if query parameters are not properly escaped.

Mitigation: Already uses parameterized queries so this is correct. This can be moved to SQLAlchemy ORM but not necessary.

Affected files:

- flaskshop/public/views.py:53

B704 - Markupsafe XSS (MEDIUM Severity)

Issue: Uses `Markup()` which disables auto-escaping. Cross-site scripting (XSS) if untrusted data is passed to `Markup()`.

Mitigation: Sanitize `result` before being passed to `Markup()`.

Affected files:

- flaskshop/plugin/utils.py:38

B113 - Request without Timeout (MEDIUM Severity)

Issue: Makes HTTP request without timeout. Application hangs indefinitely if remote server is unresponsive.

Mitigation: This is an exploit file with no relation to the main codebase so it is acceptable. Add timeout parameter `requests.get(url, timeout=10)`.

Affected files:

- exploits/task2-4.py:18

B311 - Random Usage (LOW Severity, 13 issues)

Issue: Standard pseudo-random generators are not suitable for security purposes

Mitigation: Use `secrets` package for security-critical operations (passwords, tokens). Keep `random` for non-security purposes like in `random_data.py:501`.

Affected files:

- flaskshop/account/utils.py:159
- flaskshop/corelib/utils.py:28
- flaskshop/discount/models.py:52
- flaskshop/random_data.py:45, 241, 285, 295, 306, 501, 518, 535, 552, 580

B101 - Assert Used (LOW Severity, 11 issues)

Issue: Use of `assert` statements in test files.

Mitigation: This usage is acceptable in test files. Do not add asserts in production code.

Affected files:

- tests/test_config.py:10, 11, 12, 18
- tests/test_database.py:29, 35, 42, 59, 68
- tests/test_e2e.py:6, 13

B105, B106 - Hardcoded Passwords (LOW Severity, 5 issues)

Issue: Hardcoded passwords in test data and settings. Credentials exposed in source code, especially PayPal secret in settings file.

Mitigation: Move sensitive credentials to environment variables or `.env` file.

Affected files:

- flaskshop/settings.py:89 (B105 - PayPal secret)
- tests/settings.py:7 (B105)
- flaskshop/random_data.py:356, 388, 396, 399 (B106 - 4 issues)

B403 - Pickle Import (LOW Severity)

Issue: Pickle package can execute arbitrary code during deserialization.

Mitigation: This is a false positive. Only the error type is imported, not the package utils.

Affected files:

- flaskshop/corelib/mc.py:3

Recommendations Priority

This priority is based on the impact of the issues and their context grouping.

1. High Priority:

- B602 (shell injection)
- B404/B603 (subprocess with static values),

2. Medium Priority:

- B704 (XSS)

3. Low Priority:

- B113 (timeout)
- B105/B106 (hardcoded passwords)
- B311 (random)

4. Acceptable:

- B608 (SQL)
- B324 (weak hash)
- B101 (asserts in tests)